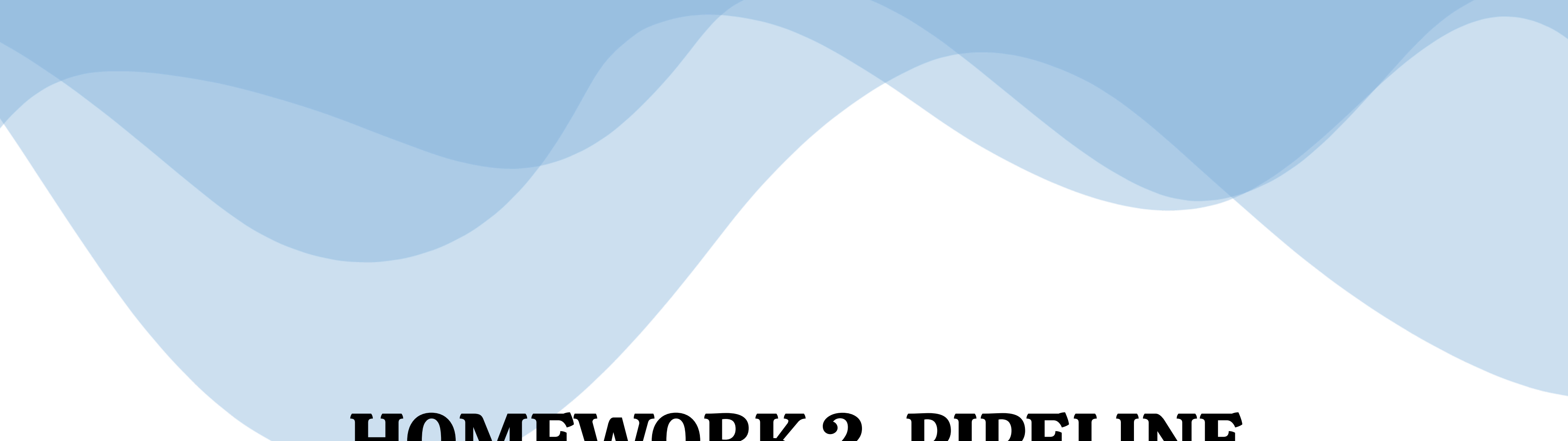


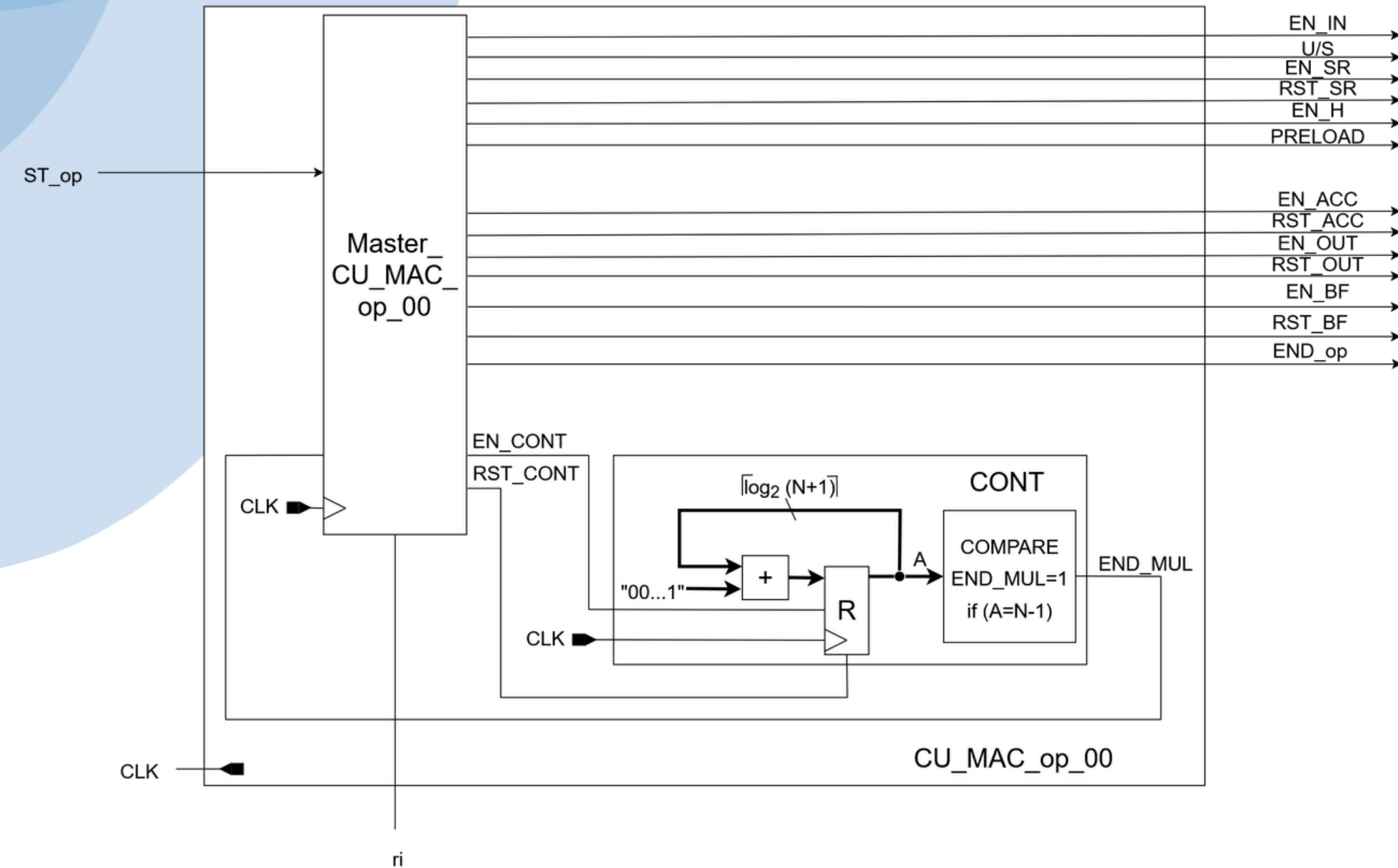


HOMEWORK 2-PIPELINE

CRISTINA VALLESI RAIMONDI

DSPHW A.Y. 2025/2026

MAC_op_00



L'obiettivo del progetto è quello di modificare la versione precedente del Filtro FIR con MAC sequenziale, introducendo buffer e pipeline.

La struttura esterna, ovvero quella che si occupa dell'interazione con l'ambiente circostante, è stata mantenuta. Mentre è stata modificata la struttura che si occupa del calcolo.

Sono stati aggiunti due segnali di controllo per la gestione del buffer:

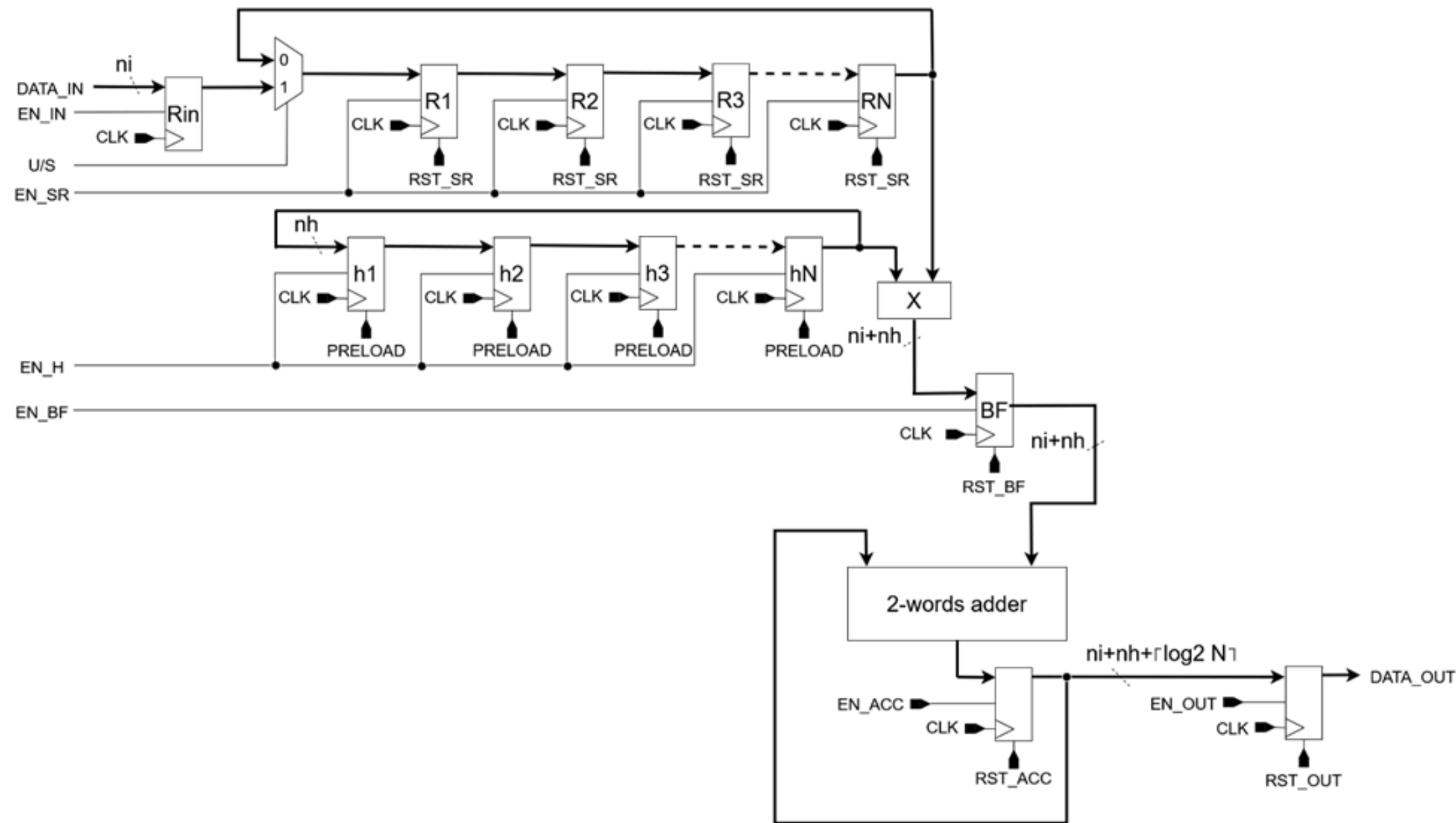
- EN_BF: abilita la memorizzazione del valore intermedio dell'operazione nel registro di buffer
- RST_BF: resetta il valore contenuto nel buffer.

La struttura interna della CU_FIRmac00 rimane invariata, mentre si è provveduto a modificare il contenuto di Core_MAC_op_00 e CU_MAC_op_00, contenuti in MAC_op_00.

Il contatore CONT è stato modificato. Nella versione originale produceva in uscita il segnale END_MAC, il quale veniva portato a 1 quando il contatore aveva contato N operazioni MAC complete. Nella nuova versione il segnale di output è stato modificato con END_MUL, il quale conta il numero di moltiplicazioni che sono state salvate nel buffer.

Avremo in output END_MUL=1 quando il contatore raggiungerà il valore N-1, ciò è stato fatto in modo da compensare la latenza di un ciclo introdotta dalla natura sincrona del sistema.

CORE_MAC_OP_00

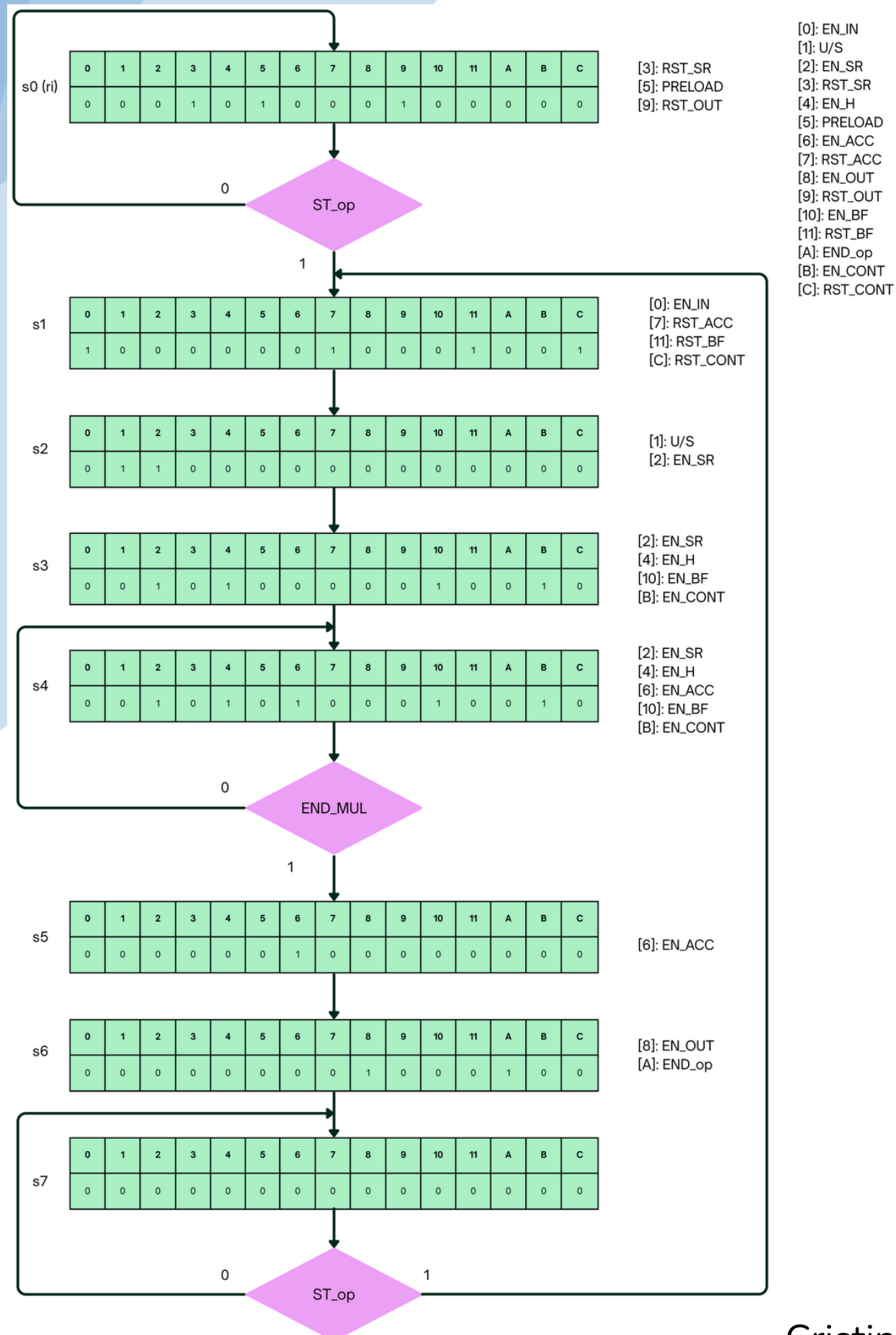


È stato inserito tra il moltiplicatore e l'accumulatore un buffer intermedio in modo da dividere il cammino critico ed aumentare la frequenza massima del clock.

Il buffer memorizzerà il risultato del prodotto quando verrà abilitato dalla CU attraverso EN_BF, contemporaneamente il contatore verrà attivato.

Poiché il contatore è aggiornato sul fronte di clock, la condizione di fine viene rilevata dalla FSM nel ciclo di clock successivo all'abilitazione di EN_CONT, garantendo così il corretto allineamento temporale tra datapath e unità di controllo.

FSM MASTER_CU_MAC_OP_00

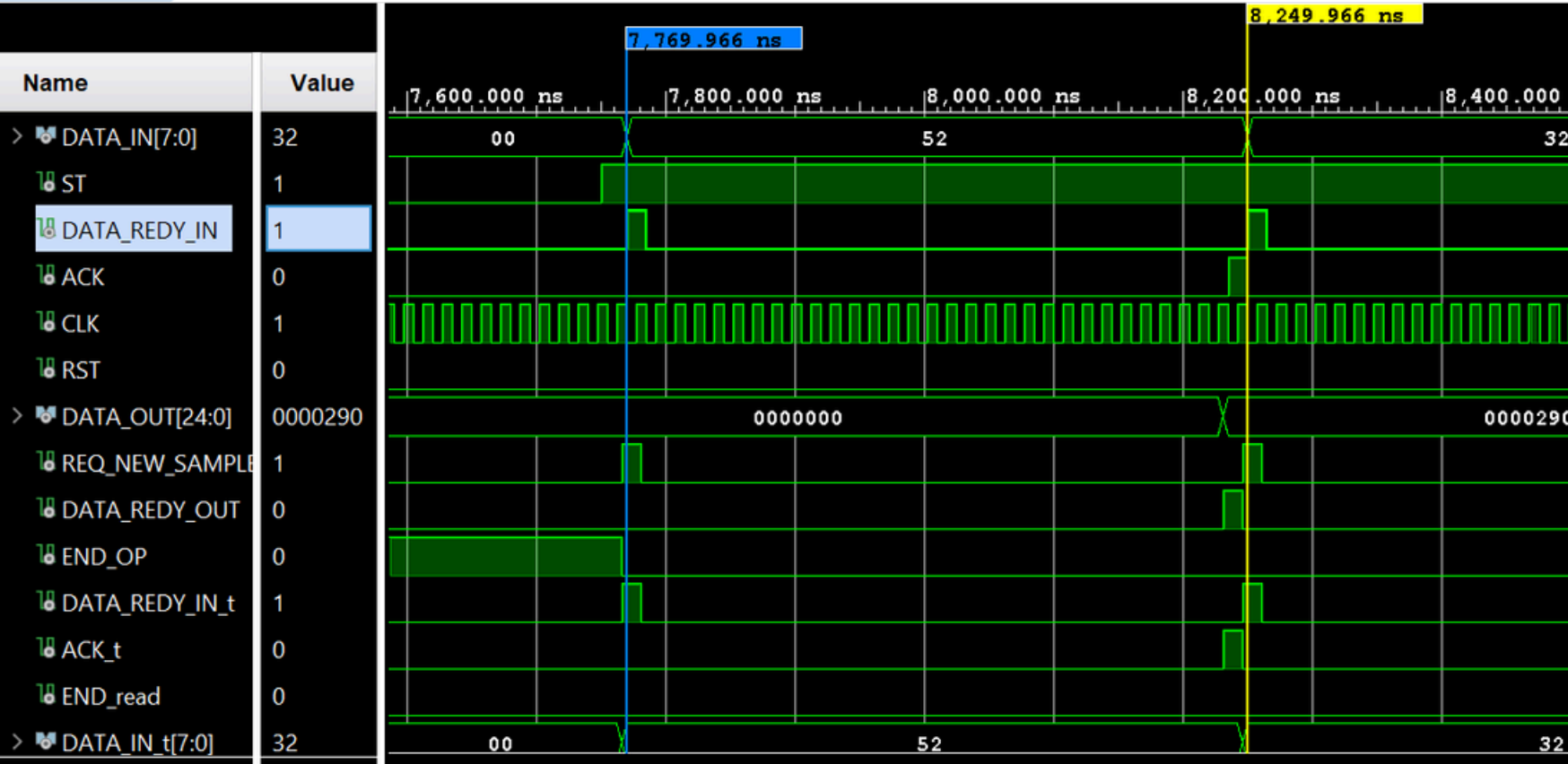
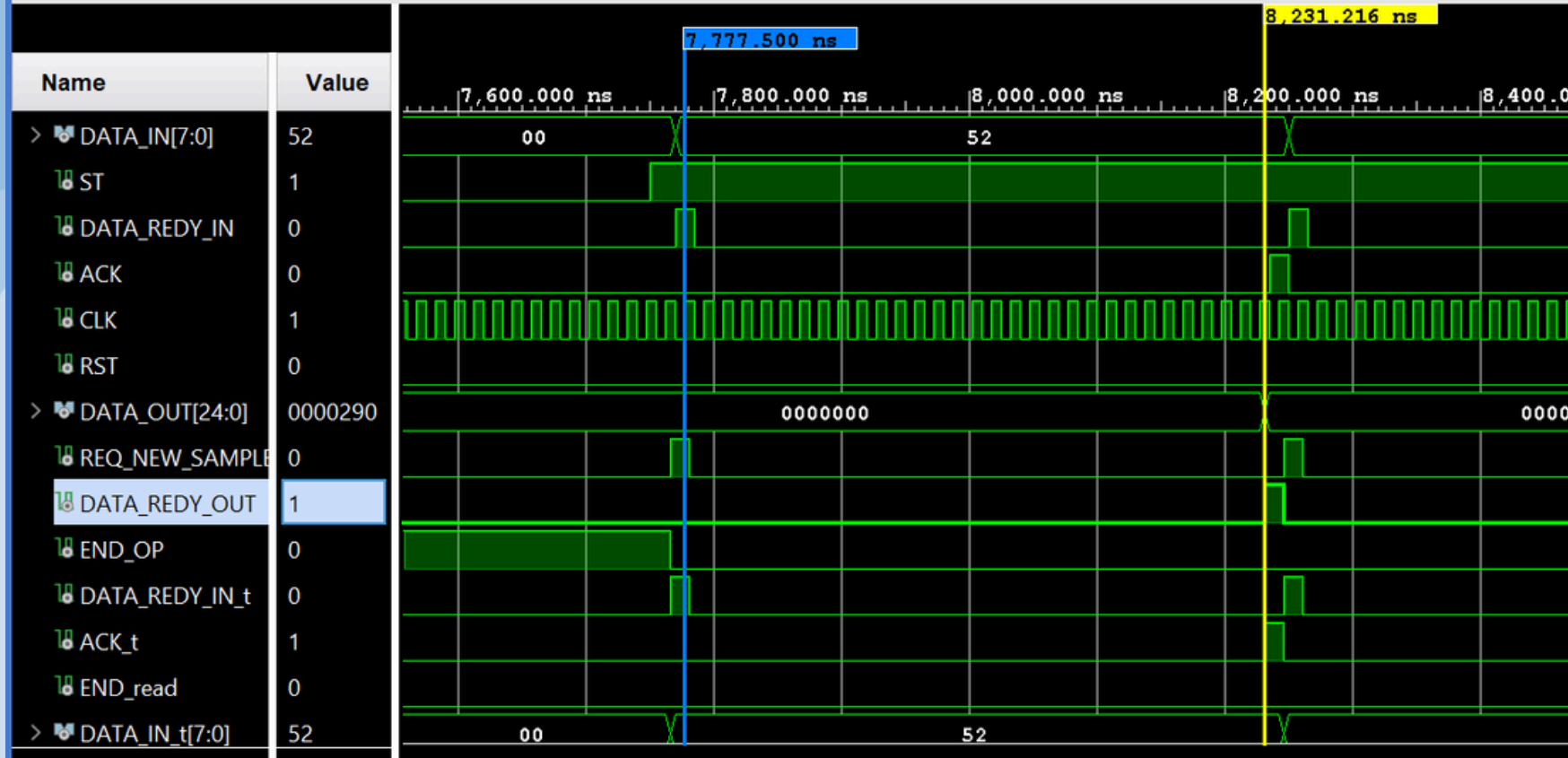


Sono stati modificati gli stati per la gestione del buffer e della pipeline:

- S3: viene attivato l'EN_BF. Il buffer memorizzerà il valore del prodotto, il ciclo dei clk successivo al calcolo del risultato, in modo che quest'ultimo sia stabile. Verranno poi fatti avanzare sia lo state register che il coefficient register.
- S4: unico stato di loop. Viene attivato il buffer per memorizzare il valore del prodotto precedente, viene attivato anche l'accumulatore che leggerà il valore "vecchio" del buffer e verranno fatti avanzare i registri. Ciò è possibile perché tutti i registri del datapath vengono aggiornati sullo stesso fronte di clock. Per passare allo stato successivo viene controllato END_MUL.
- S5: stato di flush, viene attivato l'accumulatore per effettuare l'ultimo accumulo. Dovuto all'introduzione della pipeline

Inoltre, è stato aggiunto il reset del buffer nello stato S1 dove venivano già svolti gli altri reset necessari per il corretto funzionamento del data path.

SIMULAZIONE



	FIR_MAC	FIR_MAC BF	FIR_MAC PL
Latency	76 Tclk	100 Tclk	31 Tclk
Throughput	0.855	0.654	2.08

Sono stati misurati 31 Tclk di latency tra DATA_READY_IN e DATA_READY_OUT.

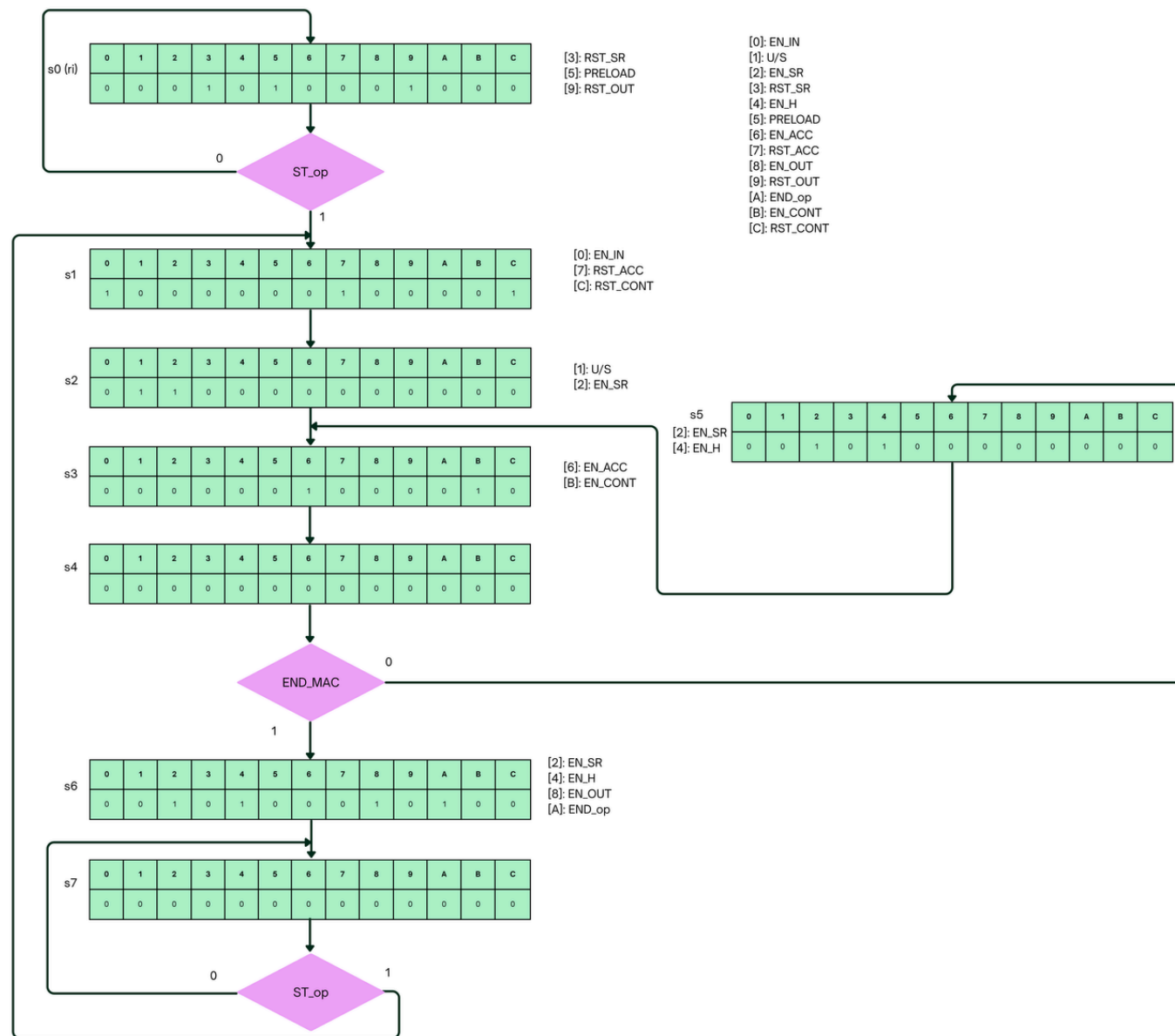
L'introduzione dello stadio di pipeline tra moltiplicatore e accumulatore ha consentito di sovrapporre le operazioni di moltiplicazione e accumulo, riducendo il tempo di elaborazione. In particolare, si misura uno speedup di 3.23x rispetto alla versione solo buffer e di 2.45x rispetto all'originale.

Sono stati misurati 32 Tclk tra due richieste consecutive di campioni, mentre nella versione senza buffer 78 Tclk e nella versione con buffer senza pileline 102.

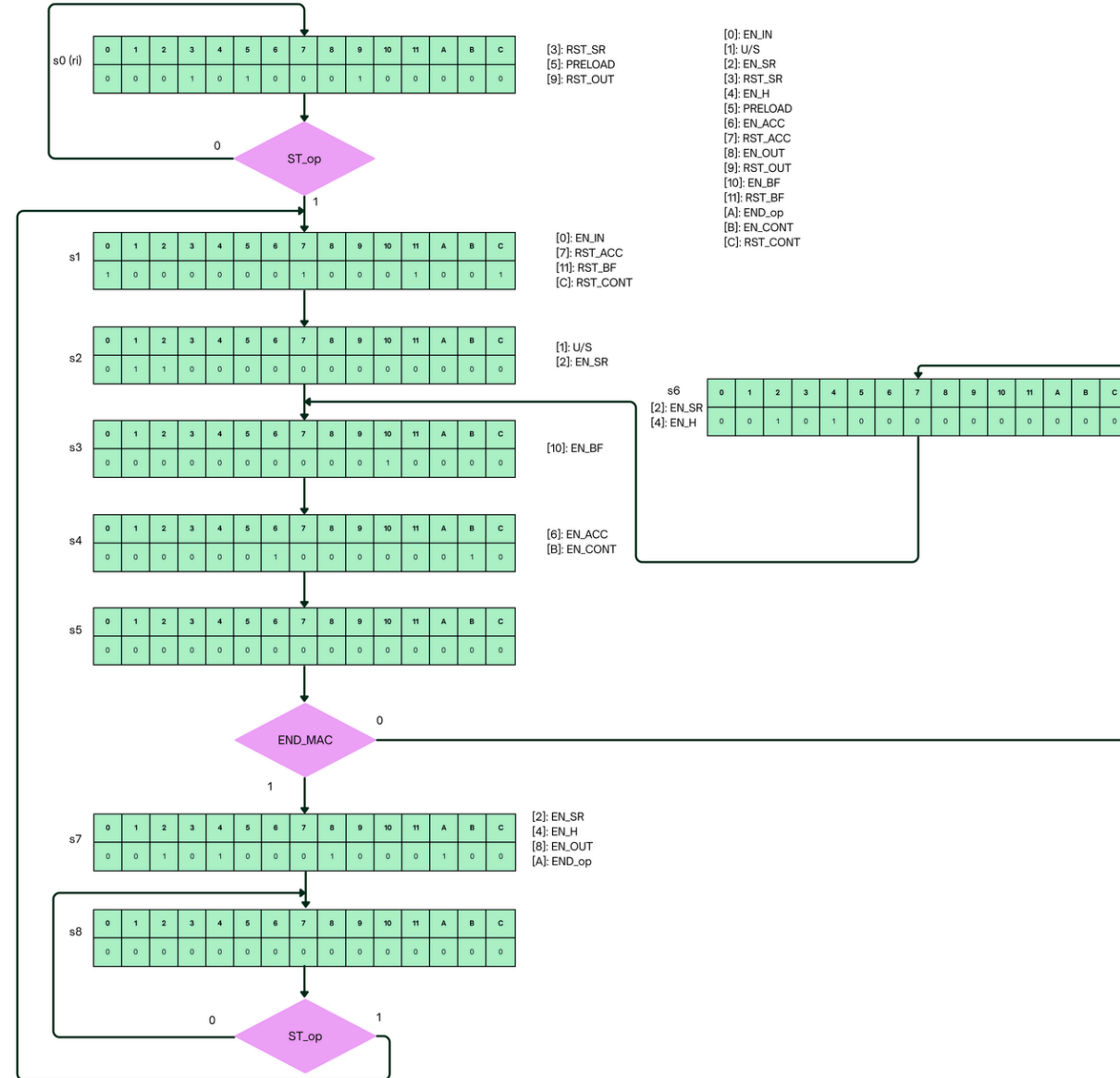
Si ottiene quindi un incremento del throughput di circa 3.2x rispetto alla versione con buffer non pipelined e di circa 2.4x rispetto alla versione originale.

CONFRONTO DELLE IMPLEMENTAZIONI

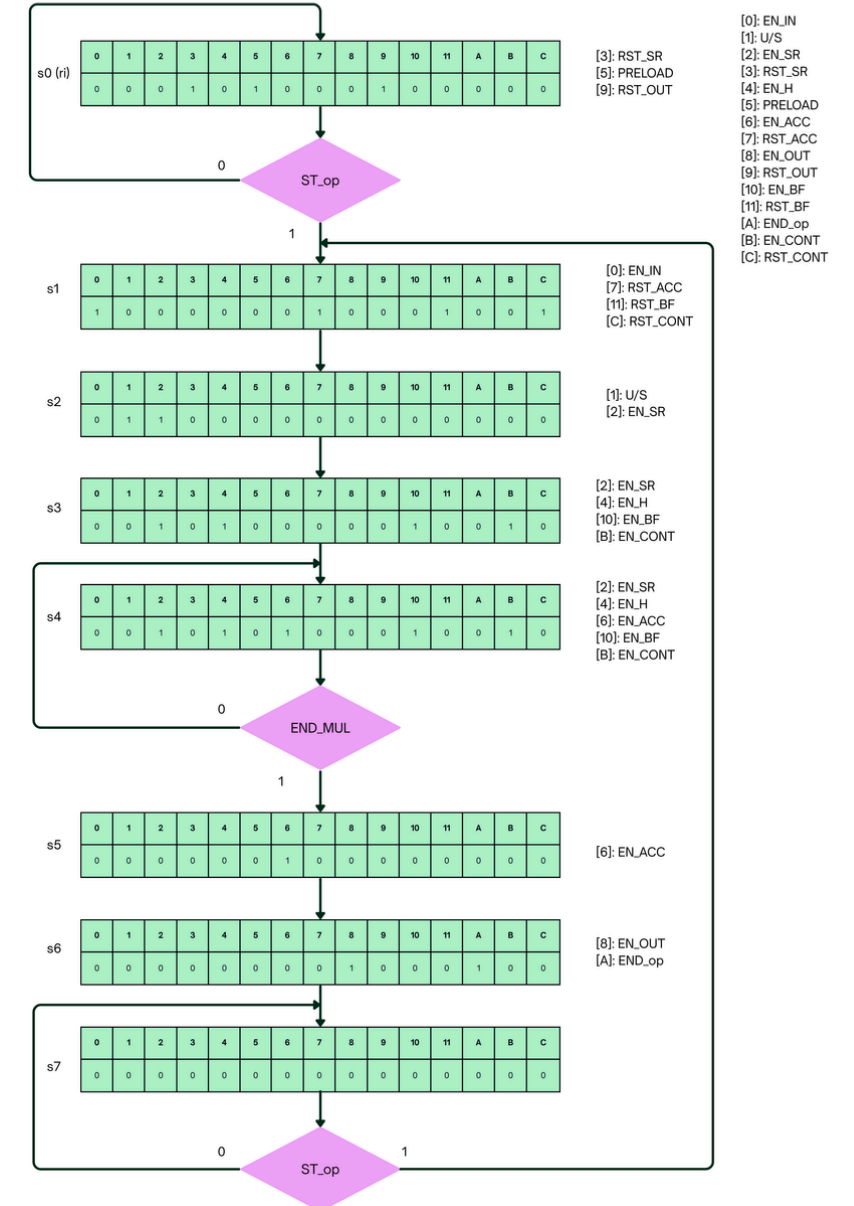
FIR_MAC



FIR_MAC BF



FIR_MAC PL



La versione originale della FSM è composta da 8 stati. Dopo l'inizializzazione e il caricamento del campione vengono eseguite in sequenza la moltiplicazione e l'accumulo.

La versione con buffer richiede invece 9 stati. L'introduzione del registro buffer tra moltiplicatore e accumulatore rende infatti necessario separare le operazioni di scrittura del buffer e di accumulo in stati distinti.

Nella versione con buffer e pipeline il numero di stati torna a 8. Prima di ciclare viene caricato il buffer con la prima moltiplicazione. Il loop viene ridotto ad uno stato all'interno del quale vengono attivati contemporaneamente buffer ed accumulatore e vengono preparati i dati per l'iterazione successiva. Nell'ultima iterazione viene effettuato l'N-1-esimo accumulo e memorizzata nel buffer l'N-esima moltiplicazione. Inoltre, i registri di campioni e coefficienti completano il loro ciclo di rotazione. Si rende pertanto necessario lo stato s5 dove si esegue l'ultimo accumulo.

CONFRONTO DELLE IMPLEMENTAZIONI

	FIR_MAC	FIR_MAC BF	FIR_MAC PL
Latency	76 Tclk	100 Tclk	31 Tclk
Throughput	0.855	0.654	2.08
WNS	9.537 ns	11.234 ns	11.462 ns
Tot on chip power	0.073 W	0.072 W	0.072 W
MAX CLK freq	183.0 MHz	265.5 MHz	280 MHz
LUT	53	53	54
FF	393	394	393
DSP	1	1	1
IO	41	41	41

Si riscontra che il solo inserimento del buffer non introduce parallelismo temporale tra le operazioni e comporta un aumento dell’overhead di controllo, con conseguente peggioramento del throughput rispetto all’architettura originale.

Ciò avviene poichè il buffer introduce un ciclo di latenza, aumentando il numero complessivo di stati e il tempo di elaborazione. Tuttavia, grazie a questa aggiunta, viene ridotto il cammino critico ed aumentata la frequenza massima del CLK.

L’introduzione della pipeline nel MAC permette di sovrapporre le operazioni di moltiplicazione e accumulo ottenendo sia una riduzione della latenza sia un incremento del throughput. Si riscontra inoltre un aumento della frequenza massima del Clk

L’architettura a pipeline rappresenta quindi la soluzione più efficiente tra quelle analizzate. Quest’ultima consente di migliorare contemporaneamente latenza, throughput e frequenza massima di funzionamento, mantenendo pressoché invariati l’occupazione di risorse hardware e il consumo di potenza.